Autonomous and Mobile Robotics

Wheeled Mobile Robots — Motion Control:
Introduction and Trajectory Tracking

Lecture notes

Prof. Giuseppe Oriolo, Sapienza Università di Roma

Contents

1 Motion Control: Setting the Problem *(slides 2–4)* **2**
1.1 Why feedback, and why a kinematic model . . . 2
1.2 Two motion control problems . . . 2

2 Trajectory Tracking: State Error Feedback *(slides 5–7)* **3**
2.1 Admissible trajectories and flatness . . . 3
2.2 The control scheme . . . 4
2.3 Error coordinates and error dynamics . . . 4

3 Approximate Linearization: Brush-up *(slides 8–9)* **5**

4 Tracking via Approximate Linearization *(slides 10–16)* **5**
4.1 Input transformation: making the origin an equilibrium . . . 5
4.2 The linear approximation . . . 6
4.3 Gain tuning . . . 6
4.4 Actual inputs, limitations and a global version . . . 7

5 Trajectory Tracking: Output Error Feedback *(slide 17)* **8**

6 Exact I/O Linearization: Brush-up *(slides 18–19)* **9**

7 Tracking via I/O Linearization: Static Feedback *(slides 20–24)* **10**
7.1 The singularity for the output (x, y) . . . 10
7.2 Displacing the output to a point B . . . 10
7.3 What happens to θ: zero dynamics . . . 11
7.4 Properties and block scheme . . . 12

8 Tracking via I/O Linearization: Dynamic Feedback *(slides 25–28)* **12**
8.1 Dynamic extension . . . 12
8.2 Control law . . . 13

9 Simulations *(slides 29–40)* **14**

10 Summary **15**

Conventions. Throughout, the WMR is a *unicycle* with configuration $\boldsymbol{q} = (x, y, \theta)^T$, Cartesian position $\boldsymbol{p} = (x, y)^T$ and kinematic model

$$\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta, \qquad \dot{\theta} = \omega, \tag{1}$$

where v is the driving velocity and ω the steering velocity. Subscript d denotes desired (reference) quantities.

1 Motion Control: Setting the Problem *(slides 2–4)*

1.1 Why feedback, and why a kinematic model

A desired motion is assigned to the WMR and the associated *nominal inputs* have been computed (e.g. by the planning techniques seen before). Applying those inputs in open loop would give very poor results: initial errors never decay, and disturbances, slippage and model errors accumulate. Hence we need *feedback control*.

For manipulators the feedback is designed on the *dynamic* model and produces commands at the generalized-force level. For WMRs one instead works on the *kinematic* model, for two reasons:

(1) the wheels are equipped with low-level control loops (PID on the actuators) that accept *velocities* as references;

(2) the dynamics is simpler than in manipulators and can be mostly canceled via feedback.

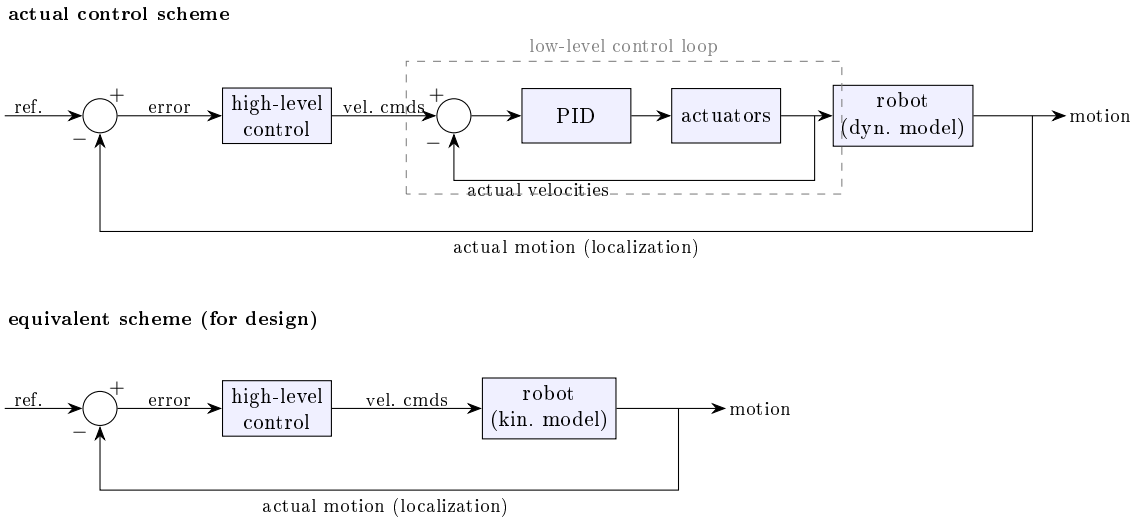


Figure 1: Actual control scheme (top) and the equivalent scheme used for design (bottom): the fast low-level loops make the velocity commands act almost instantaneously, so the robot is seen as its kinematic model.

Intuition

Think of a driver and a car. The driver (high-level control) only decides "how fast, how much steering"; the engine and steering servo (low-level loops) execute these requests much faster than the car moves on the road. So, for the purpose of planning the *path-following behaviour*, we may forget engines and masses and say that the velocities we ask for are the velocities we get. This is exactly what makes the kinematic model (1) a legitimate design model.

1.2 Two motion control problems

Definition 1.1 (Trajectory tracking and posture regulation)**.** Two standard problems:

- *Trajectory tracking*: the robot must converge to (and then follow) a desired *Cartesian trajectory* $\boldsymbol{p}_d(t)$.
- *Posture regulation*: the robot must converge to a desired *configuration* $\boldsymbol{q}_d = (x_d, y_d, \theta_d)$.

Without loss of generality we consider the unicycle in what follows. Trajectory tracking is more relevant in practice, because moving along *planned* trajectories reduces the risk of collision; on the other hand, posture regulation does not require planning.

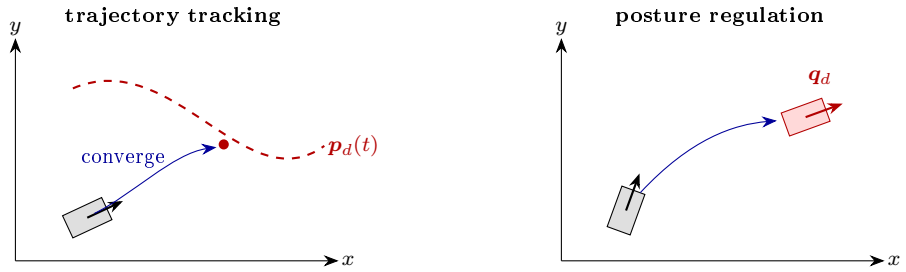


Figure 2: Tracking: follow a time-parametrized Cartesian path. Regulation: reach a final configuration (position *and* orientation), with no prescribed intermediate motion.

2 Trajectory Tracking: State Error Feedback *(slides 5–7)*

2.1 Admissible trajectories and flatness

The unicycle must track a Cartesian desired trajectory $(x_d(t), y_d(t))$ assumed to be *admissible*.

Definition 2.1 (Admissible Cartesian trajectory)**.** $(x_d(t), y_d(t))$ is admissible for the unicycle if there exist $v_d(t)$ and $\omega_d(t)$ such that

$$\dot{x}_d = v_d \cos\theta_d, \qquad \dot{y}_d = v_d \sin\theta_d, \qquad \dot{\theta}_d = \omega_d.$$

The unicycle is *differentially flat* with flat output (x, y), so the whole desired state and input trajectory can be computed algebraically from $(x_d(t), y_d(t))$ and its derivatives.

Proposition 2.2 (Flatness of the unicycle)**.** *Given an admissible* $(x_d(t), y_d(t))$ *with* $\dot{x}_d^2 + \dot{y}_d^2 \neq 0$,

$$\theta_d(t) = \mathrm{Atan2}\big(\dot{y}_d(t), \dot{x}_d(t)\big) + k\pi, \qquad k = 0, 1, \tag{2}$$

$$v_d(t) = \pm\sqrt{\dot{x}_d^2(t) + \dot{y}_d^2(t)}, \tag{3}$$

$$\omega_d(t) = \frac{\ddot{y}_d(t)\dot{x}_d(t) - \ddot{x}_d(t)\dot{y}_d(t)}{\dot{x}_d^2(t) + \dot{y}_d^2(t)}. \tag{4}$$

Proof. Dividing the second kinematic equation by the first gives $\tan\theta_d = \dot{y}_d/\dot{x}_d$, hence the expression of θ_d (the choice of k is the choice between forward and backward motion). Squaring and adding gives $v_d^2 = \dot{x}_d^2 + \dot{y}_d^2$, with the sign of v_d matched to k: $v_d > 0$ for $k = 0$, $v_d < 0$ for $k = 1$. Finally $\omega_d = \dot{\theta}_d$, and differentiating $\mathrm{Atan2}(\dot{y}_d, \dot{x}_d)$ gives $(\ddot{y}_d\dot{x}_d - \ddot{x}_d\dot{y}_d)/(\dot{x}_d^2 + \dot{y}_d^2)$, which is independent of k. □

Admissibility is guaranteed as long as $\dot{y}_d/\dot{x}_d$ is continuous in t, i.e. the tangent direction has no jumps (*no sharp corners in time*).

Intuition

A car cannot move sideways: at every instant its velocity vector must point along its heading. So if you tell me *where* the car must be at every time (the Cartesian curve), I already know *how it must be oriented* (tangent to the curve), *how fast* it goes (the speed along the curve) and *how fast it must turn* (the curvature times the speed). There are no free choices left, apart from forward/backward (k). This is flatness: the Cartesian trajectory "contains" the full state and input history. The formulas also show why a trajectory that stops ($\dot{x}_d = \dot{y}_d = 0$) is delicate: the heading is undefined there.

2.2 The control scheme

The desired state trajectory $\boldsymbol{q}_d(t) = (x_d, y_d, \theta_d)$ is used to compute the *state error*, from which the feedback action is generated; the nominal inputs (v_d, ω_d) are used as a *feedforward* term.

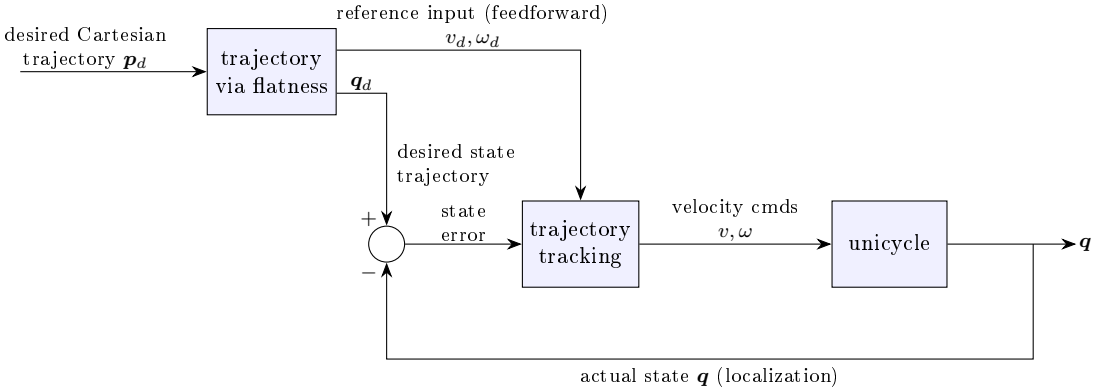


Figure 3: Trajectory tracking via state error feedback (block scheme): the desired Cartesian trajectory is turned into a desired state trajectory by flatness; the state error drives the tracking block, which also receives the nominal inputs v_d, ω_d as feedforward.

2.3 Error coordinates and error dynamics

The objective is to drive the state error to zero. Rather than $\boldsymbol{q}_d - \boldsymbol{q}$ we use its *rotated* version

$$\boldsymbol{e} = \begin{pmatrix} e_1 \\ e_2 \\ e_3 \end{pmatrix} = \begin{pmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_d - x \\ y_d - y \\ \theta_d - \theta \end{pmatrix}. \tag{5}$$

(e_1, e_2) is the Cartesian error $\boldsymbol{e}_p = \boldsymbol{p}_d - \boldsymbol{p}$ expressed in a frame rotated by θ, i.e. in the frame attached to the robot (e_1 along the sagittal axis, e_2 orthogonal to it), see Fig. 4.

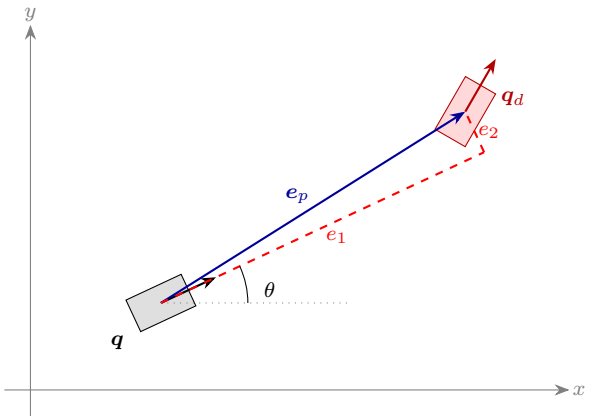


Figure 4: The rotated error: the Cartesian error $\boldsymbol{e}_p$ is decomposed along the robot's sagittal axis (e_1, "how far ahead/behind is the target") and orthogonally to it (e_2, "how far to the side").

Proposition 2.3 (Error dynamics). *Along the trajectories of the unicycle* (1) *and of the reference system, the error* (5) *obeys the* nonlinear, time-varying *dynamics*

$$\dot{e}_1 = v_d \cos e_3 - v + e_2\,\omega, \tag{6}$$
$$\dot{e}_2 = v_d \sin e_3 - e_1\,\omega, \tag{7}$$
$$\dot{e}_3 = \omega_d - \omega. \tag{8}$$

Proof. Differentiate $e_1 = \cos\theta(x_d - x) + \sin\theta(y_d - y)$:

$$\dot{e}_1 = \omega\big[-\sin\theta(x_d - x) + \cos\theta(y_d - y)\big] + \cos\theta(\dot{x}_d - \dot{x}) + \sin\theta(\dot{y}_d - \dot{y}) = \omega e_2 + v_d\cos(\theta_d - \theta) - v,$$

where we used $\dot{x}_d = v_d\cos\theta_d$, $\dot{y}_d = v_d\sin\theta_d$, $\dot{x} = v\cos\theta$, $\dot{y} = v\sin\theta$ and $\cos\theta\cos\theta_d + \sin\theta\sin\theta_d = \cos(\theta_d - \theta)$. The computation of $\dot{e}_2$ is analogous, using $-\sin\theta\cos\theta_d + \cos\theta\sin\theta_d = \sin(\theta_d - \theta)$, and $\dot{e}_3 = \omega_d - \omega$ is immediate. □

Intuition

The term $e_2\omega$ in $\dot{e}_1$ (and $-e_1\omega$ in $\dot{e}_2$) is the price of using a *rotating* frame: when the robot turns, the frame turns with it and the components of a fixed error vector mix, exactly as the components of a fixed arrow rotate when you spin your coordinate axes. In exchange, the other terms are very readable: $v_d\cos e_3 - v$ says "if I drive slower than the reference speed (projected on my heading), the target runs away from me along e_1".

3 Approximate Linearization: Brush-up *(slides 8–9)*

The idea: to stabilize a nonlinear system at an equilibrium, stabilize its *approximate linearization* around it. Consider

$$\dot{\boldsymbol{x}} = \boldsymbol{\eta}(\boldsymbol{x}, \boldsymbol{u}) = \boldsymbol{f}(\boldsymbol{x}) + \boldsymbol{G}(\boldsymbol{x})\boldsymbol{u}, \qquad \boldsymbol{x} \in \mathbb{R}^n,\ \boldsymbol{u} \in \mathbb{R}^m,$$

and assume (w.l.o.g.) that the origin is the desired *unforced* equilibrium: $\boldsymbol{\eta}(\boldsymbol{0}, \boldsymbol{0}) = \boldsymbol{0}$, or equivalently $\boldsymbol{f}(\boldsymbol{0}) = \boldsymbol{0}$. The Taylor expansion of $\boldsymbol{\eta}$ around $\boldsymbol{x} = \boldsymbol{0}, \boldsymbol{u} = \boldsymbol{0}$ gives

$$\boldsymbol{\eta}(\boldsymbol{x}, \boldsymbol{u}) \approx \cancel{\boldsymbol{\eta}(\boldsymbol{0}, \boldsymbol{0})} + \left.\frac{\partial \boldsymbol{\eta}}{\partial \boldsymbol{x}}\right|_{\substack{\boldsymbol{x}=\boldsymbol{0}\\ \boldsymbol{u}=\boldsymbol{0}}} \boldsymbol{x} + \left.\frac{\partial \boldsymbol{\eta}}{\partial \boldsymbol{u}}\right|_{\substack{\boldsymbol{x}=\boldsymbol{0}\\ \boldsymbol{u}=\boldsymbol{0}}} \boldsymbol{u}.$$

The *approximate linearization* at the origin is then

$$\dot{\boldsymbol{x}} = \boldsymbol{A}\boldsymbol{x} + \boldsymbol{B}\boldsymbol{u}, \qquad \boldsymbol{A} = \left.\frac{\partial \boldsymbol{\eta}}{\partial \boldsymbol{x}}\right|_{\boldsymbol{0},\boldsymbol{0}}, \quad \boldsymbol{B} = \left.\frac{\partial \boldsymbol{\eta}}{\partial \boldsymbol{u}}\right|_{\boldsymbol{0},\boldsymbol{0}}.$$

Let $\boldsymbol{u} = \boldsymbol{K}\boldsymbol{x}$, so that $\dot{\boldsymbol{x}} = (\boldsymbol{A} + \boldsymbol{B}\boldsymbol{K})\boldsymbol{x}$. If $\boldsymbol{K}$ is chosen so that $\boldsymbol{A} + \boldsymbol{B}\boldsymbol{K}$ is *Hurwitz* (certainly possible if $(\boldsymbol{A}, \boldsymbol{B})$ is controllable), the approximate linearization is asymptotically stable.

Theorem 3.1 (Lyapunov indirect method)**.** *If $\boldsymbol{A} + \boldsymbol{B}\boldsymbol{K}$ is Hurwitz, the origin is a* locally *asymptotically stable equilibrium of the nonlinear closed-loop system $\dot{\boldsymbol{x}} = \boldsymbol{\eta}(\boldsymbol{x}, \boldsymbol{K}\boldsymbol{x})$. (If $\boldsymbol{A} + \boldsymbol{B}\boldsymbol{K}$ has an eigenvalue with positive real part the origin is unstable; if the eigenvalues are only marginally stable, the linearization is inconclusive.)*

Intuition

Zoom in enough on any smooth function and it looks like a straight line (here, a linear map). If the linear caricature of the system is stable, then close enough to the equilibrium the true system behaves like it and is stable too. The word *local* is essential: the linear model says nothing about what happens far away, where the curvature (the neglected nonlinear terms) matters. That is exactly why this method will only give local guarantees for the unicycle.

4 Tracking via Approximate Linearization *(slides 10–16)*

4.1 Input transformation: making the origin an equilibrium

Apply the approximate linearization approach to the error dynamics of Proposition 2.3. In that form the origin is *not* an unforced equilibrium (even with $\boldsymbol{e} = \boldsymbol{0}$, the terms $v_d - v$ and $\omega_d - \omega$

would not vanish unless the inputs equal the reference). This is rectified by the invertible *input transformation*

$$w_1 = v_d \cos e_3 - v, \qquad w_2 = \omega_d - \omega. \tag{9}$$

The error dynamics $\dot{\boldsymbol{e}} = \boldsymbol{\eta}(\boldsymbol{e}, \boldsymbol{w})$ becomes

$$\dot{\boldsymbol{e}} = \underbrace{\begin{pmatrix} \omega_d e_2 \\ -\omega_d e_1 + v_d \sin e_3 \\ 0 \end{pmatrix}}_{\boldsymbol{f}(\boldsymbol{e})} + \underbrace{\begin{pmatrix} 1 & -e_2 \\ 0 & e_1 \\ 0 & 1 \end{pmatrix}}_{\boldsymbol{G}(\boldsymbol{e})} \begin{pmatrix} w_1 \\ w_2 \end{pmatrix}, \tag{10}$$

which is at rest when $\boldsymbol{e} = \boldsymbol{0}, \boldsymbol{w} = \boldsymbol{0}$, i.e. $\boldsymbol{\eta}(\boldsymbol{0}, \boldsymbol{0}) = \boldsymbol{0}$. Both the drift and the input term are nonlinear; in general the drift is also time-varying (through $v_d(t), \omega_d(t)$).

Intuition

The new inputs $\boldsymbol{w}$ are simply "*how far the actual velocities deviate from what is needed to keep the error at rest*". $w_1 = 0$ means we drive at the speed that makes e_1 stand still; $w_2 = 0$ means we turn at the reference rate. Feedback will generate $\boldsymbol{w}$ from the error; the actual commands (v, ω) are recovered afterwards by inverting (9). So the feedforward (v_d, ω_d) is automatically included.

4.2 The linear approximation

One easily finds

$$\frac{\partial \boldsymbol{\eta}}{\partial \boldsymbol{e}} = \begin{pmatrix} 0 & \omega_d - w_2 & 0 \\ -\omega_d + w_2 & 0 & v_d \cos e_3 \\ 0 & 0 & 0 \end{pmatrix}, \qquad \frac{\partial \boldsymbol{\eta}}{\partial \boldsymbol{w}} = \begin{pmatrix} 1 & -e_2 \\ 0 & e_1 \\ 0 & 1 \end{pmatrix}.$$

Letting $\boldsymbol{e} = \boldsymbol{0}, \boldsymbol{w} = \boldsymbol{0}$ gives

$$\boldsymbol{A}(t) = \begin{pmatrix} 0 & \omega_d & 0 \\ -\omega_d & 0 & v_d \\ 0 & 0 & 0 \end{pmatrix}, \qquad \boldsymbol{B} = \begin{pmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{pmatrix}.$$

Note that $\boldsymbol{A}$ is actually $\boldsymbol{A}(t)$ because of the dependence of v_d and ω_d on time $\Rightarrow$ the linear approximation is still a *time-varying* system!

Define the *linear* feedback

$$\boldsymbol{w} = \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} = \begin{pmatrix} -k_1 & 0 & 0 \\ 0 & -k_2 & -k_3 \end{pmatrix} \begin{pmatrix} e_1 \\ e_2 \\ e_3 \end{pmatrix} = \boldsymbol{K}\boldsymbol{e}.$$

The closed-loop error dynamics is still time-varying:

$$\dot{\boldsymbol{e}} = (\boldsymbol{A}(t) + \boldsymbol{B}\boldsymbol{K})\boldsymbol{e} = \boldsymbol{A}_{cl}(t)\boldsymbol{e} = \begin{pmatrix} -k_1 & \omega_d & 0 \\ -\omega_d & 0 & v_d \\ 0 & -k_2 & -k_3 \end{pmatrix} \boldsymbol{e}. \tag{11}$$

4.3 Gain tuning

Proposition 4.1 (Pole placement for the frozen system). *Letting*

$$k_1 = k_3 = 2\zeta a, \qquad k_2 = \frac{a^2 - \omega_d^2}{v_d}, \qquad a > 0,\ \zeta \in (0, 1),$$

the characteristic polynomial of $\boldsymbol{A}_{cl}(t)$ becomes time-invariant and Hurwitz:

$$p(\lambda) = (\lambda + 2\zeta a)(\lambda^2 + 2\zeta a\lambda + a^2).$$

Proof. Expanding $\det(\lambda \boldsymbol{I} - \boldsymbol{A}_{cl})$ along the first row,

$$\det \begin{pmatrix} \lambda + k_1 & -\omega_d & 0 \\ \omega_d & \lambda & -v_d \\ 0 & k_2 & \lambda + k_3 \end{pmatrix} = (\lambda + k_1)\big(\lambda^2 + k_3\lambda + v_d k_2\big) + \omega_d^2(\lambda + k_3).$$

With $k_1 = k_3 = 2\zeta a$ and $v_d k_2 = a^2 - \omega_d^2$ this becomes $(\lambda+2\zeta a)\big(\lambda^2+2\zeta a\lambda+a^2-\omega_d^2\big)+\omega_d^2(\lambda+2\zeta a) = (\lambda + 2\zeta a)(\lambda^2 + 2\zeta a\lambda + a^2)$. □

The polynomial has *one real negative eigenvalue* $-2\zeta a$ and *two complex eigenvalues with damping* ζ *and natural frequency* a, $\lambda = -\zeta a \pm ja\sqrt{1-\zeta^2}$.

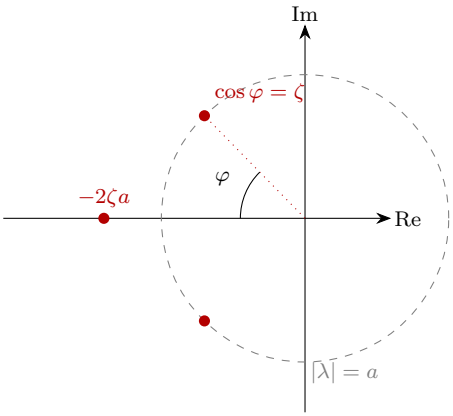


Figure 5: Closed-loop eigenvalues for $a = 2$, $\zeta = 0.7$: a real pole at $-2\zeta a$ and a complex pair on the circle of radius a with damping ζ (the angle φ from the negative real axis satisfies $\cos\varphi = \zeta$).

Intuition

The two gains k_1, k_3 are chosen equal to the damping term and k_2 absorbs the (time-varying) reference velocities v_d, ω_d. The trick is "gain scheduling": we *re-compute* k_2 *at every instant* so that the frozen-time eigenvalues stay at the same place. The designer then simply chooses the response speed a and damping ζ, as for a second-order system. Note the price: $k_2 = (a^2 - \omega_d^2)/v_d \to \infty$ as $v_d \to 0$.

Remark (Caveat). Having time-invariant, Hurwitz *frozen-time* eigenvalues does *not* guarantee asymptotic stability of a time-varying system. Stability is guaranteed only if v_d and ω_d are constant ($\Rightarrow \boldsymbol{A}_{cl}$ constant), i.e. for rectilinear and circular reference trajectories, and then Theorem 3.1 gives only *local* asymptotic stability, not global.

4.4 Actual inputs, limitations and a global version

The actual velocity inputs (v, ω) are obtained by plugging the feedback controls w_1, w_2 in the input transformation (9):

$$v = v_d \cos e_3 - w_1 = v_d \cos e_3 + k_1 e_1, \qquad \omega = \omega_d - w_2 = \omega_d + k_2 e_2 + k_3 e_3. \tag{12}$$

Two remarks:

- $(v, \omega) \to (v_d, \omega_d)$ as $\boldsymbol{e} \to \boldsymbol{0}$: the controller reduces to pure feedforward.
- $k_2 \to \infty$ as $v_d \to 0$, so this controller can only be used with *persistent* Cartesian trajectories (stops are not allowed).

Theorem 4.2 (Global stability via a nonlinear version)**.** *Global asymptotic stability of* $\boldsymbol{e} = \mathbf{0}$ *is obtained by the nonlinear feedback*

$$w_1 = -k_1(v_d, \omega_d)\, e_1, \tag{13}$$

$$w_2 = -k_2\, v_d\, \frac{\sin e_3}{e_3}\, e_2 - k_3(v_d, \omega_d)\, e_3, \tag{14}$$

if k_1, k_3 *are bounded, positive, with bounded derivatives (and* $k_2 > 0$ *constant), and* v_d, ω_d *are bounded, with* v_d *not vanishing asymptotically.*

Intuition

In the linearized controller the term $k_2 v_d e_2$ is linear in e_3-space, i.e. it approximates $\sin e_3$ by e_3. The nonlinear version keeps $\sin e_3 / e_3$, which is bounded (between -0.22 and 1 for all e_3): large heading errors can no longer produce unreasonably large commands, and a Lyapunov argument (not repeated here) works globally. The structure is the same, and when e_3 is small the two controllers coincide.

Algorithm 4.3 (Tracking via state error feedback and approximate linearization)**.** At each control instant:

1. from (x_d, y_d) compute θ_d, v_d, ω_d via flatness (Prop. 2.2);
2. read (x, y, θ) from localization and compute $\boldsymbol{e}$ with (5);
3. compute gains $k_1 = k_3 = 2\zeta a$, $k_2 = (a^2 - \omega_d^2)/v_d$;
4. set $v = v_d \cos e_3 + k_1 e_1$ and $\omega = \omega_d + k_2 e_2 + k_3 e_3$.

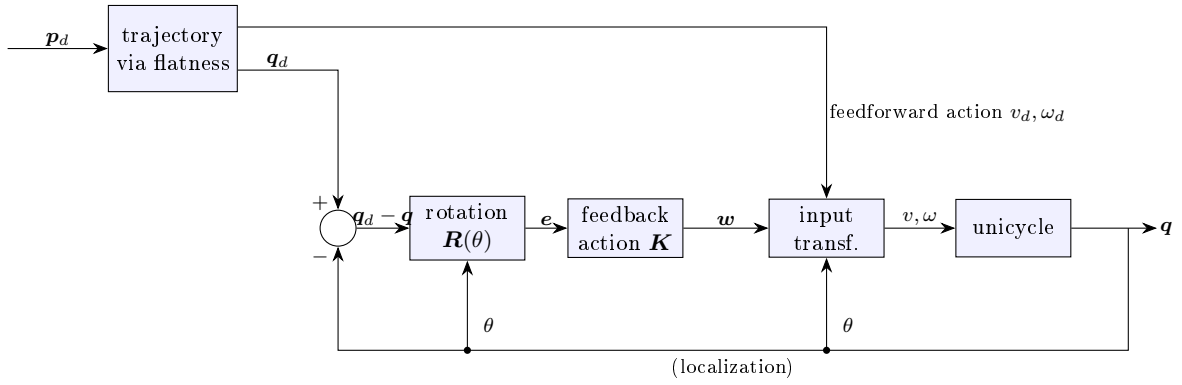


Figure 6: Detailed version of Fig. 3 for approximate linearization: a static controller on the rotated state error, with feedforward v_d, ω_d added through the input transformation.

The final scheme (Fig. 6) is a *static* controller based on the state error; it needs v_d, ω_d, and needs θ both for the error rotation and for the input transformation.

5 Trajectory Tracking: Output Error Feedback *(slide 17)*

Another approach: develop the feedback action from the *output (Cartesian) error* only, *without* computing a desired state trajectory, using as *feedforward* term the velocity $\dot{\boldsymbol{p}}_d$ along the reference trajectory.

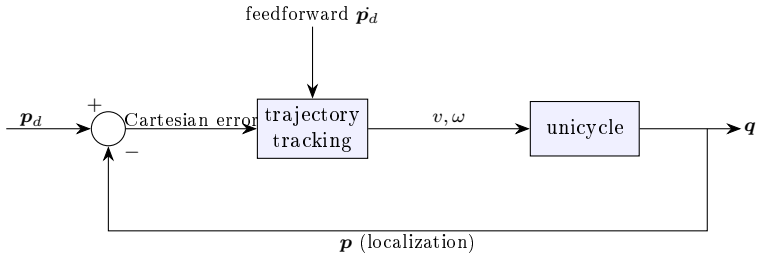


Figure 7: Output error feedback: only the Cartesian error is fed back.

The difficulty is that the unicycle has two inputs and, if only (x, y) is to be controlled, a standard linearization-by-feedback argument must be adapted. The next section recalls the tool, and the following ones apply it in two different ways.

6 Exact I/O Linearization: Brush-up *(slides 18–19)*

Consider a driftless nonlinear system with as many outputs as inputs:

$$\dot{\boldsymbol{x}} = \boldsymbol{G}(\boldsymbol{x})\boldsymbol{u}, \qquad \boldsymbol{y} = \boldsymbol{h}(\boldsymbol{x}), \qquad \boldsymbol{x} \in \mathbb{R}^n,\ \boldsymbol{u} \in \mathbb{R}^m,\ \boldsymbol{y} \in \mathbb{R}^m.$$

Since

$$\dot{\boldsymbol{y}} = \frac{\partial \boldsymbol{h}}{\partial \boldsymbol{x}}\dot{\boldsymbol{x}} = \underbrace{\frac{\partial \boldsymbol{h}}{\partial \boldsymbol{x}}\boldsymbol{G}(\boldsymbol{x})}_{\boldsymbol{T}(\boldsymbol{x})}\boldsymbol{u} = \boldsymbol{T}(\boldsymbol{x})\boldsymbol{u},$$

we call $\boldsymbol{T}$ $(m \times m)$ the *decoupling matrix*.

Proposition 6.1 (Exact input–output linearization)**.** *If $\boldsymbol{T}(\boldsymbol{x})$ is invertible, the* linearizing feedback $\boldsymbol{u} = \boldsymbol{T}^{-1}(\boldsymbol{x})\boldsymbol{w}$ *yields*

$$\dot{\boldsymbol{y}} = \boldsymbol{T}(\boldsymbol{x})\boldsymbol{T}^{-1}(\boldsymbol{x})\boldsymbol{w} = \boldsymbol{w},$$

i.e. an exactly linear (and decoupled) map between the new inputs $\boldsymbol{w}$ and the time derivative of the outputs. Given a reference output $\boldsymbol{y}_d(t)$, the output error $\boldsymbol{e} = \boldsymbol{y}_d - \boldsymbol{y}$ has dynamics $\dot{\boldsymbol{e}} = \dot{\boldsymbol{y}}_d - \boldsymbol{w}$; the law $\boldsymbol{w} = \dot{\boldsymbol{y}}_d + \boldsymbol{K}\boldsymbol{e}$ (feedforward + proportional feedback) gives $\dot{\boldsymbol{e}} = -\boldsymbol{K}\boldsymbol{e}$, i.e. global exponential stability *of $\boldsymbol{e} = \boldsymbol{0}$ provided that the eigenvalues of $\boldsymbol{K}$ are in the right half-plane. The final control law is*

$$\boldsymbol{u} = \boldsymbol{T}^{-1}(\boldsymbol{x})\big(\dot{\boldsymbol{y}}_d + \boldsymbol{K}\boldsymbol{e}\big). \tag{15}$$

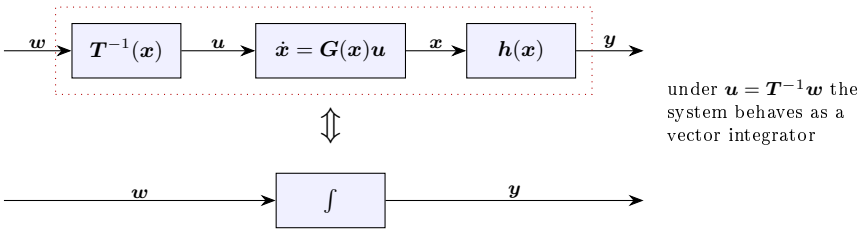


Figure 8: Exact input–output linearization: the dotted block, seen from $\boldsymbol{w}$ to $\boldsymbol{y}$, is just a bank of integrators.

Intuition

Imagine a robot arm where you want the hand velocity $\dot{\boldsymbol{y}}$ and you can command joint velocities $\boldsymbol{u}$ through a Jacobian-like matrix $\boldsymbol{T}(\boldsymbol{x})$. If you multiply the desired hand velocity by the inverse Jacobian, the hand does exactly what you wanted, and the plant "disappears": what remains is $\dot{y} = w$, the simplest system in the world. Tracking becomes trivial: ask for the reference velocity plus a correction proportional to the error. Everything hinges on $\boldsymbol{T}$ being *invertible*.

7 Tracking via I/O Linearization: Static Feedback *(slides 20–24)*

7.1 The singularity for the output (x, y)

Let us apply exact i/o linearization to the unicycle with output (x, y). However, the decoupling matrix is

$$\begin{pmatrix} \dot{x} \\ \dot{y} \end{pmatrix} = \begin{pmatrix} \cos\theta & 0 \\ \sin\theta & 0 \end{pmatrix} \begin{pmatrix} v \\ \omega \end{pmatrix},$$

which is *singular* (its second column is zero): ω does not appear in the first derivative of (x, y). Hence exact input–output linearization is *not possible* for the output (x, y) by static feedback.

Intuition

At the contact point of the wheel, turning in place does not move the point. The steering command is invisible at the level of positions, so we cannot "invert" the map. The cure is to look at a point that *does* move when you steer.

7.2 Displacing the output to a point B

Solution: *change slightly the output* so that the new input–output map is invertible. In particular, displace the output from the contact point of the wheel to a point B along the sagittal axis, at distance b:

$$x_B = x + b\cos\theta, \qquad y_B = y + b\sin\theta.$$

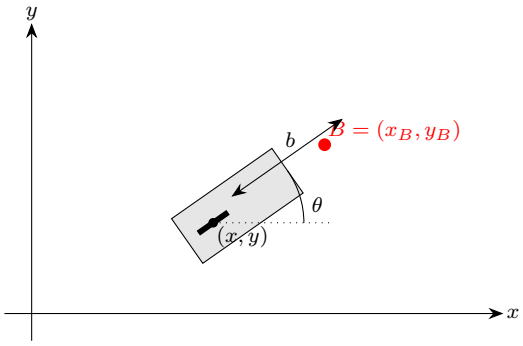


Figure 9: Point B at distance b ahead of the wheel contact point along the sagittal axis.

Differentiating with respect to time,

$$\begin{pmatrix} \dot{x}_B \\ \dot{y}_B \end{pmatrix} = \underbrace{\begin{pmatrix} \cos\theta & -b\sin\theta \\ \sin\theta & b\cos\theta \end{pmatrix}}_{\boldsymbol{T}(\theta),\ \det = b} \begin{pmatrix} v \\ \omega \end{pmatrix}.$$

If $b \neq 0$ we may set

$$\begin{pmatrix} v \\ \omega \end{pmatrix} = \boldsymbol{T}^{-1}(\theta) \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} = \begin{pmatrix} \cos\theta & \sin\theta \\ -\sin\theta/b & \cos\theta/b \end{pmatrix} \begin{pmatrix} w_1 \\ w_2 \end{pmatrix}, \tag{16}$$

obtaining the input–output linearized system

$$\dot{x}_B = w_1, \qquad \dot{y}_B = w_2, \qquad \dot{\theta} = \frac{w_2 \cos\theta - w_1 \sin\theta}{b}.$$

(The third equation is the part of the dynamics that is *not* seen from the output.)

We achieve *global exponential convergence* of (x_B, y_B) to the desired trajectory by letting

$$w_1 = \dot{x}_d + k_x(x_d - x_B), \qquad w_2 = \dot{y}_d + k_y(y_d - y_B), \qquad k_x, k_y > 0.$$

Intuition

Hold a broom by its middle and push the handle sideways: the bristle end (B) moves sideways even if the hand does not. In the same way, a point ahead of the axle *moves laterally when the robot steers*, so both of its velocity components can be commanded; b is the lever arm, and $\det \boldsymbol{T} = b$ tells us that the shorter the lever arm, the more delicate the inversion (the $1/b$ in the second row of (16) amplifies the steering command).

7.3 What happens to θ: zero dynamics

The orientation θ is *not* controlled by this scheme, which is based on output error feedback: there is a *zero dynamics*. Still, it must evolve as dictated by flatness. The next result makes this precise.

Proposition 7.1 (Zero dynamics). *Suppose B tracks the reference exactly, so $w_1 = \dot{x}_d$, $w_2 = \dot{y}_d$. Writing $\dot{\boldsymbol{p}}_d = |\dot{\boldsymbol{p}}_d|(\cos\varphi, \sin\varphi)$ and $\delta = \theta - \varphi$,*

$$\dot{\delta} = -\frac{|\dot{\boldsymbol{p}}_d|}{b}\sin\delta + (\textit{terms in } \dot{\varphi}),$$

and for the case $\varphi = \text{const}$ (straight reference) $\delta = 0$ is exponentially stable if $|\dot{\boldsymbol{p}}_d| > 0$ and $b > 0$, while $\delta = \pi$ is unstable.

Proof. Substitute w_1, w_2 in $\dot{\theta}$: $\dot{\theta} = \big(\dot{y}_d\cos\theta - \dot{x}_d\sin\theta\big)/b = \frac{|\dot{\boldsymbol{p}}_d|}{b}\sin(\varphi - \theta)$. Subtracting $\dot{\varphi}$ gives the claim; linearization at $\delta = 0$ gives $\dot{\delta} \simeq -(|\dot{\boldsymbol{p}}_d|/b)\delta$, at $\delta = \pi$ it gives $+(|\dot{\boldsymbol{p}}_d|/b)\delta'$. □

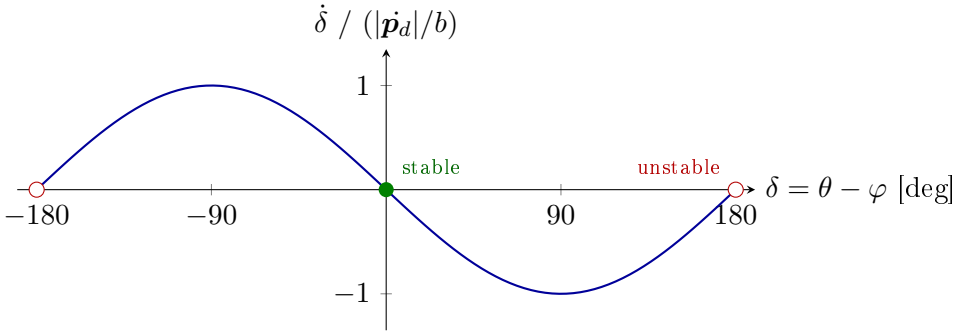


Figure 10: Phase line of the zero dynamics (straight reference): the robot's orientation relaxes to the direction of motion of B.

Intuition

It is like towing a shopping cart by its front end: if you pull B along a path, the cart (robot body) lines up behind the pulling point and follows the tangent of the path. Aligned forward ($\delta = 0$) is stable. Driving *backwards*, with the body ahead of the pulled point, is unstable: the same reason a cart pushed from behind jackknifes. This is why the scheme is used with

forward motion only.

7.4 Properties and block scheme

The desired trajectory for B can be *arbitrary*; in particular, *square corners may be included*, since no desired heading has to be computed.

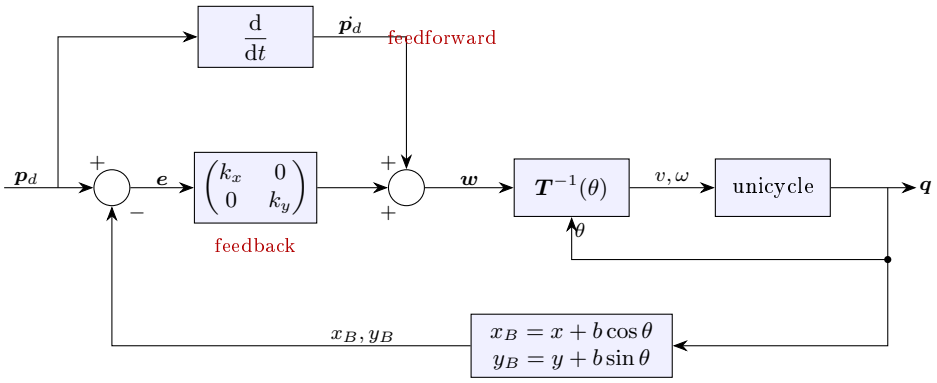


Figure 11: Tracking via output error feedback + static i/o linearization: a static controller on the output error; it needs $\dot{\boldsymbol{p}}_d$, and (x, y, θ) both for the output reconstruction and (with θ) the input transformation.

Algorithm 7.2 (Static i/o linearization tracker)**.** Given $b \neq 0$, $k_x, k_y > 0$ and the reference $\boldsymbol{p}_d(t), \dot{\boldsymbol{p}}_d(t)$:

1. compute $x_B = x + b\cos\theta$, $y_B = y + b\sin\theta$;
2. $w_1 = \dot{x}_d + k_x(x_d - x_B)$, $w_2 = \dot{y}_d + k_y(y_d - y_B)$;
3. $v = \cos\theta\, w_1 + \sin\theta\, w_2$, $\omega = (-\sin\theta\, w_1 + \cos\theta\, w_2)/b$.

Remark (Steady-state error equals b). Because B (not the contact point (x, y)) is the point forced onto $\boldsymbol{p}_d(t)$, in steady state the contact point trails the reference by b: the error $\|\boldsymbol{p}_d - \boldsymbol{p}\|$ settles at b (circle and square examples below). Decreasing b reduces this error but increases the steering velocity, as the $1/b$ in (16) shows.

8 Tracking via I/O Linearization: Dynamic Feedback *(slides 25–28)*

Rather than displacing the controlled output to B, we can *keep the (flat) output* (x, y) and achieve exact linearization by using a *dynamic* compensator.

8.1 Dynamic extension

Perform a *dynamic extension* on the driving-velocity channel ("slow down" the input): add an integrator

$$\dot{v} = a,$$

so that the new input is the driving *acceleration* a, and v becomes a state of the compensator. We can now differentiate the output further:

$$\ddot{x} = a\cos\theta - v\sin\theta\,\omega, \qquad \ddot{y} = a\sin\theta + v\cos\theta\,\omega.$$

The new decoupling matrix is nonsingular if $v \neq 0$:

$$\begin{pmatrix} \ddot{x} \\ \ddot{y} \end{pmatrix} = \underbrace{\begin{pmatrix} \cos\theta & -v\sin\theta \\ \sin\theta & v\cos\theta \end{pmatrix}}_{\boldsymbol{T}(v,\theta),\ \det = v} \begin{pmatrix} a \\ \omega \end{pmatrix}.$$

Under this assumption we may set

$$\begin{pmatrix} a \\ \omega \end{pmatrix} = \boldsymbol{T}^{-1}(\theta, v) \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} = \begin{pmatrix} \cos\theta & \sin\theta \\ -\dfrac{\sin\theta}{v} & \dfrac{\cos\theta}{v} \end{pmatrix} \begin{pmatrix} w_1 \\ w_2 \end{pmatrix},$$

obtaining a *fully linearized* system $\ddot{x} = w_1$, $\ddot{y} = w_2$ (two chains of two integrators; the order, 4, equals the dimension of the extended state (x, y, θ, v), so nothing is left unseen).

Intuition

Why did ω vanish from $\dot{x}, \dot{y}$ but appear in $\ddot{x}, \ddot{y}$? Because steering changes the *direction* of the velocity, and direction becomes visible one differentiation later, as an acceleration *multiplied by the current speed* v. If $v = 0$ turning does nothing to the position, hence the singularity at $v = 0$. Using an integrator on v is just a way to let the compensator "remember" the current speed, so that it can be divided by.

8.2 Control law

We achieve *global exponential convergence* of (x, y) to the desired trajectory by letting

$$w_1 = \ddot{x}_d + k_{p,x}(x_d - x) + k_{d,x}(\dot{x}_d - \dot{x}), \tag{17}$$
$$w_2 = \ddot{y}_d + k_{p,y}(y_d - y) + k_{d,y}(\dot{y}_d - \dot{y}), \tag{18}$$

with $k_{px}, k_{py}, k_{dx}, k_{dy} > 0$. The error then satisfies $\ddot{e} + k_d\dot{e} + k_p e = 0$ along each axis (e.g. $k_p = a^2$, $k_d = 2\zeta a$ for a prescribed damping and bandwidth).

Properties:

- since the original flat output is controlled, there is *no zero dynamics* with this approach;
- the desired trajectory must be *twice differentiable* and *persistent*, i.e. $v_d \neq 0$ always (this singularity is *structural* in nonholonomic systems).

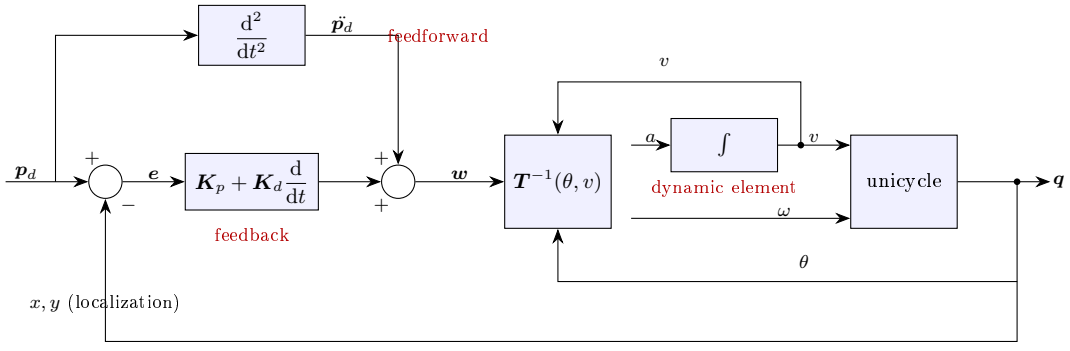


Figure 12: Tracking via output error feedback + dynamic i/o linearization: a *dynamic* controller based on the output error; it needs $\dot{\boldsymbol{p}}_d$ and $\ddot{\boldsymbol{p}}_d$ (inside $\boldsymbol{K}_d$ and the feedforward), (x, y) for the error and θ for the input transformation. The state v of the integrator is fed back to $\boldsymbol{T}^{-1}$.

Algorithm 8.1 (Dynamic i/o linearization tracker)**.** Initialize $v(0) \neq 0$ (e.g. $v(0) = v_d(0)$). At each instant:

1. compute w_1, w_2 as above (with $\dot{x} = v\cos\theta$, $\dot{y} = v\sin\theta$);

2. $a = \cos\theta\, w_1 + \sin\theta\, w_2$, $\omega = (-\sin\theta\, w_1 + \cos\theta\, w_2)/v$;
3. integrate $\dot{v} = a$ and send (v, ω) to the robot.

Intuition

Compared with the static scheme, we pay with extra machinery (one integrator, second derivatives of the reference, $v \neq 0$) but we get the real thing: the point that converges is the actual position (x, y), with zero steady-state error and no hidden heading dynamics. Static feedback tracks a *fictitious* point B; dynamic feedback tracks the *true* point.

9 Simulations *(slides 29–40)*

The slides compare the three controllers on three reference trajectories: a circle, a square (with sharp corners) and a figure-8. The qualitative findings can be summarized as follows.

	Approx. linearization	**Static i/o lin.** ($b = 0.75$ / 0.2)	**Dynamic i/o lin.**
Circle	converges; only local stability guaranteed	steady-state error $= b$; reducing b lowers the error but *increases steering velocity*	zero steady-state error, reasonable velocities
Square	only local stability; *a new transient at each corner*	error $= b$; the displaced output gives a *lookahead* behaviour (corners are cut); small b: smaller error, larger steering peaks	faster transients, reasonable velocities
Figure 8	local stability not guaranteed (persistent v_d, ω_d vary), but good performance	error $= b$; smaller $b \Rightarrow$ larger steering velocity	zero steady-state error, reasonable velocities

Table 1: Qualitative outcome of the simulations.

Reading off the circle plots, the *initial* steering velocity grows from about 2.7 rad/s for $b = 0.75$ to about 10 rad/s for $b = 0.2$: a ratio $\approx 3.7 \approx 0.75/0.2$, as expected from the $1/b$ factor in (16).

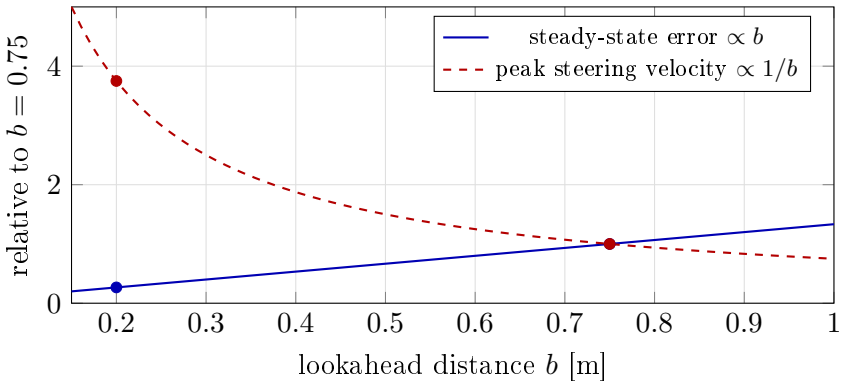


Figure 13: The trade-off of static i/o linearization (original illustration of the scaling laws; the dots mark the two cases simulated, $b = 0.75$ and $b = 0.2$). A small b gives small error but demands large steering velocities.

Example 9.1 (Circle tracking, independent simulation)**.** To visualize the comparison we simulated (Euler, $dt = 2$ ms) the tracking of a circle of radius 3 m with $v_d = 1$ m/s ($\omega_d =$

1/3 rad/s), starting from $\boldsymbol{q}(0) = (1, -1, 2)$ and, for the dynamic controller, $v(0) = 1.5$ m/s. Gains: approximate linearization $a = 2, \zeta = 0.7$; static i/o $k_x = k_y = 2.5$; dynamic i/o $k_p = 6.25$, $k_d = 5$. Fig. 14 shows the paths and the error norm.

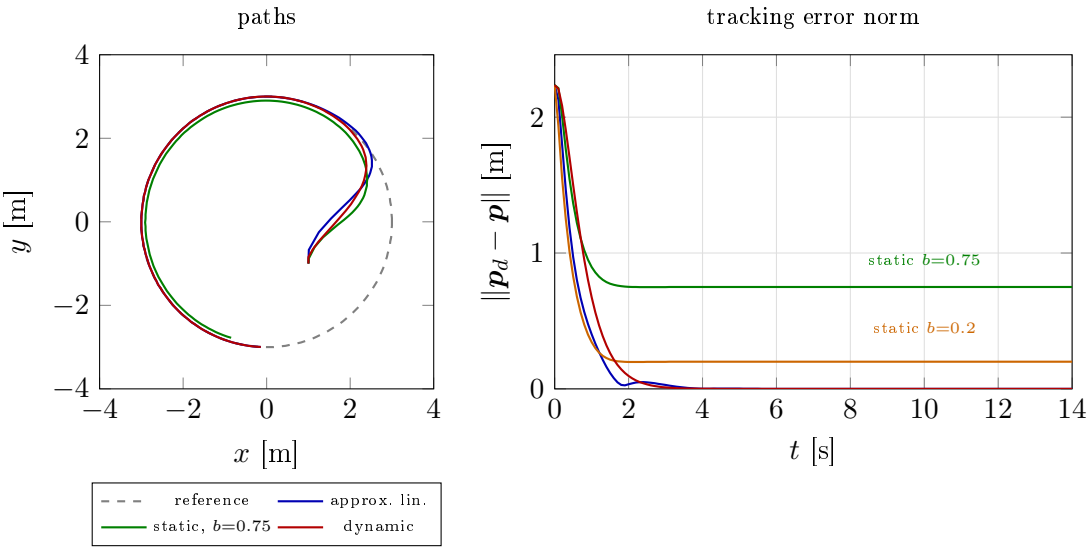


Figure 14: Original simulation of the three controllers on a circle. Approximate linearization and dynamic i/o linearization converge to zero error; static i/o linearization settles at an error equal to b.

10 Summary

	State error + approx. linearization	**Output error + static i/o lin.**	**Output error + dynamic i/o lin.**
Controlled variable	state error $\boldsymbol{e} = \boldsymbol{R}(\theta)(\boldsymbol{q}_d - \boldsymbol{q})$	Cartesian position of B (displaced by b)	Cartesian position (x, y)
Controller type	static	static	dynamic (integrator on v)
Reference needed	v_d, ω_d (and θ_d) via flatness	$\boldsymbol{p}_d, \dot{\boldsymbol{p}}_d$	$\boldsymbol{p}_d, \dot{\boldsymbol{p}}_d, \ddot{\boldsymbol{p}}_d$
Measurements	x, y, θ	x, y, θ	x, y, θ (and v internal)
Stability	local, only for constant v_d, ω_d (global with nonlinear version)	global exponential for B	global exponential for (x, y)
Zero dynamics	none (θ in the error)	yes: θ aligns with motion (forward only)	none
Steady-state error	0	b (contact point)	0
Restrictions	persistent trajectory ($k_2 \to \infty$ as $v_d \to 0$); smooth direction	none on corners; $b \neq 0$; large ω if b small	persistent ($v \neq 0$); trajectory twice differentiable

Table 2: Comparison of the trajectory tracking controllers for the unicycle.

Intuition

Pick the tool by what you can accept. *State error + linearization*: simplest idea, but the guarantees are weak. *Static i/o*: guaranteed and simple, but you pay with a permanent offset b (a lookahead, which is sometimes even desirable on corners). *Dynamic i/o*: the "right" answer (true position, zero error, globally exponentially stable) at the cost of more machinery and a ban on stopping. The trade-off between offset and steering effort in the static case is

shown again in Fig. 13.

Key take-aways

1. WMR motion control is designed on the *kinematic* model, since the fast low-level loops let us treat velocities as inputs; reference inputs must always be applied together with a feedback correction.

2. For an admissible Cartesian trajectory, *flatness* gives θ_d, v_d, ω_d algebraically, and the nominal inputs act as feedforward in all schemes.

3. State error feedback with approximate linearization has a time-varying closed loop: pole placement with $k_2 = (a^2 - \omega_d^2)/v_d$ is only rigorous for circles and lines (local stability), and requires $v_d \neq 0$; a nonlinear variant gives global stability.

4. Exact i/o linearization of (x, y) fails with static feedback (singular decoupling matrix); displacing the output to B at distance b fixes this, at the cost of a zero dynamics and a steady-state error b with steering velocity $\propto 1/b$.

5. A dynamic extension ($\dot{v} = a$) allows tracking the true flat output (x, y) with global exponential convergence and zero steady-state error, but needs a twice-differentiable, persistent trajectory ($v \neq 0$, a structural limitation of nonholonomic systems).